Beyond Semantic IDs: Finite-Beam Output Support in Generative Recommendation

Anonymous submission

Abstract

Generative recommendation (GR) retrieves items by autoregressively decoding discrete Semantic IDs (SIDs) over learned codebooks, typically with finite-width beam search. We study how this practical search budget distributes retrieval support across items. We distinguish three quantities that are often conflated: whether an item has a unique code, whether it can be scored by an interface, and whether it actually appears in a finite top-K output. To isolate the output interface, we introduce two continuous-query methods in a shared frozen item space: CQG-Single generates one query in a single forward pass, while CQG-AR autoregressively generates continuous residuals that refine the query. Both remove the item-token vocabulary and retrieve by dense scoring or approximate nearest-neighbor search. Controlled diagnostics reveal a checkpoint- and budget-specific finite-beam reachability ceiling and, in some popularity strata and evaluated checkpoints, a depth-dependent pattern in which SID decoding can be competitive at shallow K while continuous retrieval becomes more favorable deeper in the ranked list. Under a unified seven-seed ML-1M protocol with valid-prefix constrained TIGER decoding, CQG-Single obtains R@10 0.3104 ± 0.0047, compared with 0.2996 ± 0.0072 for TIGER and 0.2976 ± 0.0032 for CQG-AR (mean $\pm$ sample SD). CQG-Single exceeds TIGER in all seven indexed runs (mean paired difference 0.0107; exact sign-flip $p = .0156$). CQG-AR provides an architecture-matched test of whether autoregression requires discrete SID outputs. Our results motivate returned-list-size-matched evaluation of discrete and continuous retrieval interfaces rather than treating catalog-wide scoreability as empirical top-K coverage.

Introduction

Generative recommendation (GR) retrieves items by generating symbolic identifiers rather than directly scoring the entire catalog. The most influential instance of this paradigm is Semantic-ID (SID) recommendation: items are first quantized into discrete code sequences, and an autoregressive model decodes those sequences conditioned on user attributes and interaction history (Rajput et al. 2023; Zheng et al. 2024; Wang et al. 2024). This formulation converts recommendation into a sequence-generation problem, and supports constrained decoding through a trie, ensuring that the generated token sequence corresponds to a valid item.

This marks a substantial architectural shift from conventional sequential recommenders. Models such as SASRec encode user histories, user features, and item representations, then score candidate items discriminatively (Kang and McAuley 2018). In SID-based generative recommendation, the output interface is instead moved into a language-model-style generator: a decoder is trained to emit the SID sequence. Larger pretrained language models can provide useful generic knowledge and may benefit from scaling behavior as model and data size increase (Kaplan et al. 2020), but SID-based GR still requires three additional system components: First, items must be compressed into a finite hierarchical identifier space, using vector-quantization methods such as VQ-VAE (van den Oord, Vinyals, and Kavukcuoglu 2017), residual quantization as in RQ-VAE (Lee et al. 2022), residual or balanced hierarchical clustering, or industrial generative retrieval designs such as OneRec (Deng et al. 2025). Second, the generator must be trained either by adapting a pretrained LLM through continued pretraining and supervised fine-tuning, where SID tokens become a new vocabulary that must be aligned with the LLM's semantic space, or by training a smaller decoder-only model from scratch, which sacrifices pretrained knowledge for a cleaner SID prediction objective. Third, inference decodes SID sequences from user features and sequential history, then maps the generated codes to item IDs or expands them through item-to-item retrieval. (Figure 1)

These steps introduce two practical bottlenecks. The first is coverage and updateability: a catalog item is useful only if its SID can be generated and mapped back to the item, but newly created items may cause semantic drift between the clustering model labeling and the item meaning. The second is support imbalance: a finite hierarchical SID space is a compromise for representing an open-ended catalog. Items are not distributed uniformly across SID leaves, semantic partitions can overlap, and the generated SID set may fail to recall enough valid items even when the related item catalog itself is

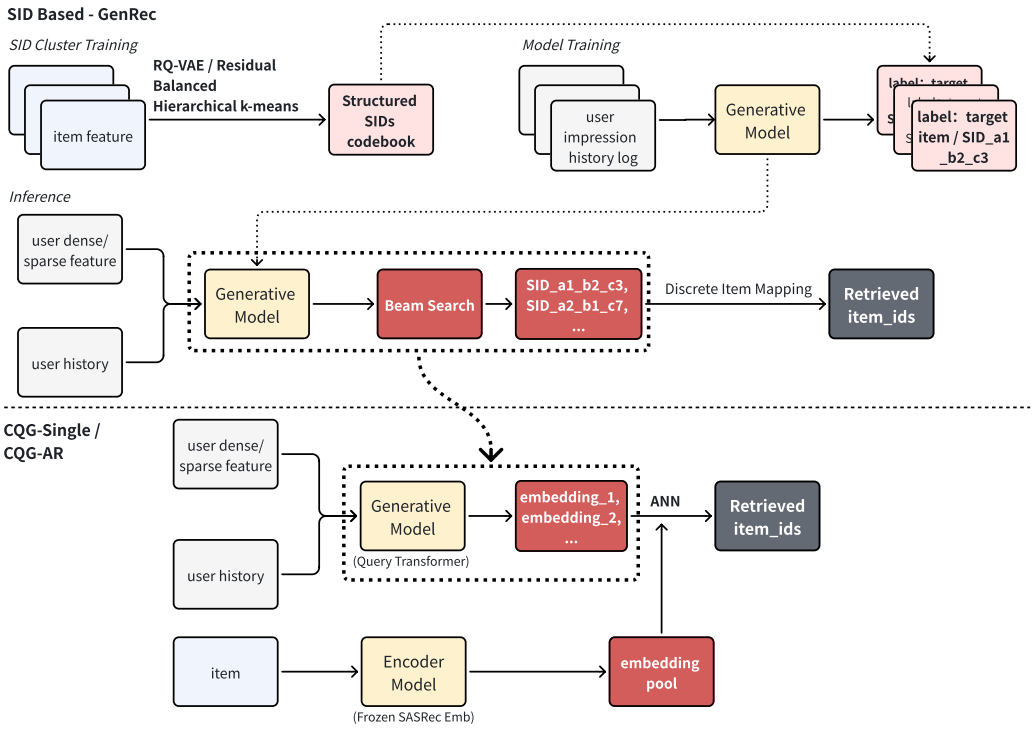

Figure 1: Inference interfaces for SID retrieval, CQG-Single, and CQG-AR. SID retrieval autoregressively generates discrete code tokens and explores candidate paths with finite-width beam search. CQG-Single produces one continuous query in a single forward pass, whereas CQG-AR autoregressively generates residual updates that refine a continuous query. Both CQG variants retrieve from the same frozen item embedding index by dense scoring or ANN, without an item-token vocabulary or code-to-item lookup.

large.

Because SID inference explores only a prefix-limited subset of the code tree, we define a checkpoint- and budget-specific finite-beam reachability ceiling as the target-support upper bound induced by a fixed trained model and finite beam budget, even when the held-out item has a valid code.

This support limitation motivates a checkpoint- and popularity-dependent crossover pattern. At shallow retrieval depth, SID models can perform well because autoregressive decoding concentrates probability mass on high-confidence paths. On rare ML-1M targets for the evaluated seed-42 checkpoints, TIGER outperforms a continuous alternative at R@10; at some deeper K, the continuous model catches up or overtakes. We treat this as an operating-point-dependent empirical pattern, not as proof that a per-user hit rate equals an item-level catalog-coverage ceiling.

To isolate this mechanism, we study two complementary continuous-query formulations. CQG-Single is a deliberately minimal vocabulary-free model that maps user context to one continuous query in a frozen item space. CQG-AR retains a TIGER-like autoregressive decoder but replaces SID-token prediction with continuous residual generation, feeding each generated residual back to refine the query. Both retrieve by dense scoring or approximate nearest-neighbor (ANN) search without a code-to-item lookup. CQG-Single tests the simplest continuous interface, while CQG-AR controls for autoregressive model structure.

Our contributions are:

- Finite-search diagnostics. For a fixed checkpoint and search budget, we distinguish unique-code assignment, target-conditioned beam support, catalog-wide scoreability, and empirical union-of-top-K coverage, and stratify the output-based quantities by item popularity.
- Conditional crossover pattern. In some popularity strata and evaluated checkpoints, SID decoding can achieve higher shallow Recall@K, while continuous retrieval can overtake at deeper K under a matched returned-list size.
- Parallel continuous-query methods. We introduce CQG-Single, a one-step continuous retriever, and CQG-AR, an autoregressive continuous decoder. Their comparison separates the effect of autoregression from the effect of a discrete SID vocabulary.
- Controlled empirical study. Across ML-1M, Amazon Beauty, and Amazon Sports, we analyze retrieval

quality, reranked performance, loss functions, LLM backbone failures, and tokenization instability while limiting scalability claims to the tested catalog sizes.

Related Work

Semantic-ID generative recommendation. TIGER (Rajput et al. 2023) introduced hierarchical Semantic IDs using residual vector quantization and autoregressive decoding. Later work improves identifier learning and alignment through collaborative semantics, textual IDs, differentiable indexing, or soft identifiers (Zheng et al. 2024; Wang et al. 2024; Tan et al. 2024; Fu et al. 2026; Li et al. 2026). These methods differ in code construction but retain a structured output interface that typically uses constrained autoregressive decoding. We study how finite-width search allocates support over that interface, rather than proposing another discrete tokenizer. This distinction also separates successful code assignment from successful code retrieval at inference time.

Finite search support and popularity. Autoregressive tree generation can impose correlation constraints on representable item distributions (Hou et al. 2026), while training-side studies note that tail tokens receive fewer updates. We examine the complementary inference-side question: even with a valid, nearly unique code, does a target enter a finite beam? We measure target-conditioned beam support, returned-list-size-matched catalog coverage, and popularity-stratified behavior, carefully separating these quantities. In particular, a high unique-code rate does not imply that every target enters a user's beam, and a target-conditioned hit rate does not equal catalog coverage. Our experiments treat these as distinct diagnostics of a trained model and its search budget.

Vocabulary-free and continuous recommendation. DiffuRec, DreamRec, and ContRec use diffusion or continuous latent generation (Li, Sun, and Li 2023; Yang et al. 2023; Qu et al. 2026), while GPT4Rec generates textual retrieval queries (Li et al. 2023). CQG-Single provides a minimal continuous-query control, and CQG-AR adds step-wise residual refinement. Their purpose is to separate autoregression from discrete SID output, not to claim dense retrieval itself as novel. This paired design asks whether step-wise generation requires discrete identifier tokens: CQG-Single removes both the token vocabulary and serial decoding, whereas CQG-AR preserves serial generation but replaces tokens with continuous residual updates.

Sequential, dense, and hybrid retrieval. GRU4Rec, SASRec, and BERT4Rec score catalog items from interaction histories (Hidasi et al. 2016; Kang and McAuley 2018; Sun et al. 2019); dual encoders are established in retrieval and recommendation (Huang et al. 2013; Covington, Adams, and Sargin 2016). Generative retrieval instead decodes identifiers (Tay et al. 2022; Wang et al. 2022), and LIGER combines dense and generative retrieval (Yang et al. 2024). These lines establish both catalog scoring and identifier decoding as known retrieval interfaces. Our contribution is therefore a controlled comparison in a shared frozen item space, using CQG-Single and CQG-AR to distinguish finite SID output support from autoregressive modeling without attributing the comparison to a larger item encoder or an additional reranker.

Method

Problem Formulation

Let $\mathcal{I}$ be a catalog of N items and let user u have a chronological interaction sequence $s_u = (i_1, \ldots, i_t)$. The task is to retrieve the next item i_{t+1}. SID-based GR maps each item i to a discrete code sequence $z_i = (z_{i,1}, \ldots, z_{i,L})$ and trains an autoregressive model $p(z_i \mid s_u)$. Inference uses beam search over valid code prefixes.

Both of our continuous methods assume a frozen item embedding matrix $E \in \mathbb{R}^{N \times d}$, where each row e_i is an L2-normalized item vector. They condition on the same user context c_u available to the SID decoder, including the sequential item history and, when available, demographic features. They differ in whether the continuous output is produced in one step or autoregressively.

CQG-Single: Single-Step Continuous Query Generation

CQG-Single learns a single-step query generator f_θ:

$$q_u = f_\theta(c_u), \quad q_u \in \mathbb{R}^d. \tag{1}$$

The recommendation score is cosine similarity:

$$\text{score}(u, i) = q_u^\top e_i. \tag{2}$$

The top-K items are retrieved by exact dense scoring or ANN search. Every item in E is scoreable by construction, although catalog-wide scoreability does not guarantee that every item will empirically appear in a finite top-K list.

The query generator is a causal Transformer that maps the final valid history state back into the shared item space. Unlike SID decoding, it produces the retrieval query in one forward pass without an item-token vocabulary.

CQG-AR: Autoregressive Continuous Query Generation

CQG-AR retains an autoregressive decoder but replaces discrete SID-vocabulary prediction with continuous residual generation. At decoding step ℓ, the decoder conditions on the user context and all previous continuous outputs:

$$h_\ell = D_\phi(c_u, \Delta_{<\ell}), \tag{3}$$

$$\Delta_\ell = W_{\text{AR}} h_\ell, \tag{4}$$

$$\hat{e}_u^{(\ell)} = \text{norm}\Big(\hat{e}_u^{(\ell-1)} + \Delta_\ell\Big). \tag{5}$$

We initialize the cumulative query as $\hat{e}_u^{(0)} = \mathbf{0}$. The generated residual Δ_ℓ is projected back into the decoder

input space and appended as the next autoregressive input. After L steps, the refined query $\hat{e}_u^{(L)}$ retrieves items from the same frozen embedding matrix E. Thus CQG-AR preserves the conditional, step-wise decoding structure of SID retrieval while removing the SID vocabulary, codebook lookup, and discrete beam expansion.

Training Objectives

We evaluate four CQG objectives: full-catalog CE, sampled CE, in-batch InfoNCE, and MSE. For CQG-Single, q_b below is the single generated query; for CQG-AR, it is the final refinement $\hat{e}_b^{(L)}$. In the primary CQG-AR configuration, only the final query receives direct CE supervision; intermediate supervision is used only in the decoding-depth ablation.

Full-catalog cross-entropy. Our primary objective treats retrieval as classification over all items:

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{B}\sum_{b=1}^{B} \log \frac{\exp(q_b^\top e_b^+/\tau)}{\sum_{j=1}^{N} \exp(q_b^\top e_j/\tau)}, \tag{6}$$

where $\tau = 0.07$. CE gives complete catalog visibility at training time but costs $O(N)$ per step.

Sampled cross-entropy. To control training-time catalog visibility, sampled CE replaces the full denominator with the positive item and a sampled set $\mathcal{S}_b$ of negatives:

$$\mathcal{L}_{\mathrm{sCE}} = -\frac{1}{B}\sum_{b=1}^{B} \log \frac{\exp(q_b^\top e_b^+/\tau)}{\exp(q_b^\top e_b^+/\tau) + \sum_{j\in\mathcal{S}_b} \exp(q_b^\top e_j/\tau)}. \tag{7}$$

The reported control uses 256 sampled negatives.

In-batch InfoNCE. InfoNCE treats the other target items in the minibatch as negatives:

$$\mathcal{L}_{\mathrm{NCE}} = -\frac{1}{B}\sum_{b=1}^{B} \log \frac{\exp(q_b^\top e_b^+/\tau)}{\sum_{j\in\mathcal{B}} \exp(q_b^\top e_j^+/\tau)}. \tag{8}$$

Mean-squared error. We also test direct embedding regression:

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{B}\sum_{b=1}^{B} \|q_b - e_b^+\|_2^2. \tag{9}$$

MSE preserves geometric alignment but yields lower retrieval accuracy than CE in our experiments. BPR is used to pretrain the shared SASRec item encoder, not as one of the reported CQG objectives.

Retrieval, Reranking, and Contrast with SID

Standalone CQG-Single ranks items according to $q_u^\top e_i$, while CQG-AR uses $(\hat{e}_u^{(L)})^\top e_i$. Exact catalog scoring can be replaced by ANN search, such as FAISS, for large catalogs. Our CQG-AR experiments retrieve only with the final query $\hat{e}_u^{(L)}$; merging candidates from intermediate or interest-specific queries is an untested extension. In our retrieve-and-rerank evaluation, the continuous generator retrieves 100 candidates, which are then reranked by a pretrained SASRec model. In contrast, SID retrieval requires codebook learning, autoregressive discrete decoding, beam search, and code-to-item lookup; a finite beam explores only a restricted subset of valid code paths (Figure 1). CQG-Single instead generates one continuous query, while CQG-AR preserves step-wise autoregressive generation with continuous outputs. Both directly query the item embedding index without introducing an item-token vocabulary.

Dataset	Users	Items	Interactions	Density
ML-1M	6,040	3,416	999K	4.84%
Beauty	22,363	12,101	198K	0.07%
Sports	21,519	10,693	∼178K	0.08%

Table 1: Dataset statistics. Item counts exclude the padding row. Density is defined as $|\mathcal{R}|/(|\mathcal{U}||\mathcal{I}|)$, where $\mathcal{R}$ is the set of observed interactions, $\mathcal{U}$ the users, and $\mathcal{I}$ the non-padding items. The processed Sports split differs from the standard 5-core split used in several published SID studies, so its results are treated as within-paper diagnostics.

Experiments

Experimental Setup

Datasets. We use MovieLens-1M (ML-1M), Amazon Beauty, and Amazon Sports. ML-1M is the primary controlled diagnostic because its catalog is small enough for exact full-catalog scoring. Beauty and Sports test cross-domain behavior. Our processed Sports split (21,519 users and 10,693 non-padding items) differs from the commonly reported 5-core split used by TIGER, LETTER, and EAGER; consequently, we do not directly compare our Sports values with their published numbers. We exclude Yelp because its SID and continuous baselines are not aligned under one evaluation protocol.

Protocol. We use leave-one-out evaluation: the last item is held out for testing, the second-to-last item for validation, and earlier interactions for training. Historical items are excluded from ranking. We report Recall@K and NDCG@K, primarily for $K \in \{10, 30, 50, 70, 90\}$ in reachability analysis and $K \in \{5, 10, 20\}$ in cross-dataset evaluation.

Baselines We compare against SASRec (Kang and McAuley 2018), TIGER-style SID retrieval (Rajput et al. 2023), and reported SID results from TIGER, LETTER, and EAGER where protocols allow contextual comparison. We also include non-learned dense retrieval baselines: random retrieval, last-item kNN, and mean-of-5 kNN.

Model training protocol. We first train a two-layer SASRec with BPR on chronological next-item pairs and select its checkpoint by full-catalog validation NDCG@10. Its 64-dimensional item embedding table

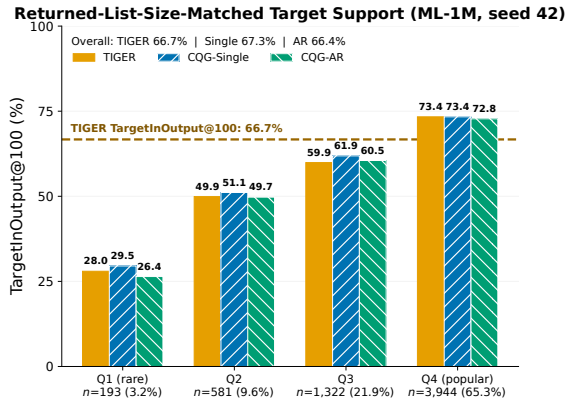

Figure 2: Returned-list-size-matched target support on ML-1M (seed 42), stratified by held-out target popularity. Bars show user-weighted TargetInOutput@100 with exactly 100 valid, distinct, history-filtered items. TIGER uses valid-prefix constrained decoding with adaptive internal over-generation; CQG-Single and CQG-AR use dense Top-100 retrieval. The dashed line marks TIGER's overall TargetInOutput@100 of 66.7%. Values are user weighted; Table 2 reports item-balanced quartile values. All analyses use the same 6,040 users and popularity-quartile mapping as Figure 3 and Table 11.

is then frozen and shared by both continuous methods as the history-input table, target embedding or classifier matrix, and retrieval index. The main experiments use no textual or language-model item encoder.

CQG-Single is a four-layer, four-head causal Transformer with width 256. It directly generates one normalized query; the final configuration uses full-catalog CE, with MSE and in-batch InfoNCE evaluated as ablations. CQG-AR uses a four-layer, six-head causal Transformer with width 384 to autoregressively generate three residual query updates. Each residual is fed into the next decoding step, while the cumulative query is updated by adding the residual and then normalizing the result. In the final configuration, only the final query receives direct full-catalog CE supervision. The multimodal Sports diagnostic instead uses a frozen fusion of collaborative and ImageNet-pretrained ResNet-18 features. Full implementation details are provided in Appendix D.

Results

Budget-Specific Finite-Beam Reachability

For a fixed model and returned-list size B, target support is the fraction of test users whose held-out target occurs anywhere in the returned list. On ML-1M, the seed-42 TargetInOutput@100 values are 66.7% for TIGER, 67.3% for CQG-Single, and 66.4% for CQG-AR. This is a per-user quantity closely related to Recall@100; it is not the fraction of catalog items retrieved. Figure 2 reports the matched popularity breakdown and user counts.

For this fixed model–search pair, TargetSupport@B is a realized ceiling on the Recall of any downstream reranker restricted to the decoded beam: a target that never enters the candidate set cannot be recovered later. This is a finite-beam reachability ceiling, not an architecture-invariant claim; increasing B can raise it. CQG avoids this particular beam-exclusion mechanism because every encoded item remains scoreable, although its finite Top-K Recall is still determined by query quality.

Table 2 provides the returned-list-size-matched finite-output comparison. TargetInOutput@100 is user weighted. Q1–Q4 first compute the hit rate of each distinct test-target item and then average items equally within training-frequency quartiles. TargetItemSupport@200 is the fraction of distinct held-out target items retrieved for at least one associated test instance. Under the corrected constrained protocol, CQG-Single retains a small advantage in overall user-weighted TargetInOutput@100 and in all four item-balanced quartiles, whereas TIGER slightly exceeds CQG-AR overall and on rare targets. Differences in distinct-target support are also small. Union-of-output catalog coverage is reported separately in Table 4. Figure 2 is user weighted, whereas the Q1–Q4 columns in Table 2 average distinct target items equally within the same quartile mapping.

This result separates code assignment from target support: 99.4% unique codes do not imply that every held-out target enters a finite returned list. Under the corrected Top-100 protocol, 33.3% of held-out user targets do not enter the corresponding candidate list; this is not a statement that 33.3% of catalog items are never retrieved. The distinct union-of-output catalog-coverage diagnostic is reported in Table 4.

Popularity-Dependent Crossover

Table 3 provides the primary cross-dataset comparison, while Figure 3 examines retrieval depth and target popularity on ML-1M; exact values and paired-bootstrap intervals are reported in Appendix C.2. Table 3 is the sole aggregate comparison. ML-1M uses the completed seeds 42–48; Beauty and Sports use seeds 42–44. The paired seed-42 analysis uses valid-prefix constrained TIGER decoding and returned lists of exactly 200 items for both methods. TIGER is slightly stronger on rare Q1 targets at several shallow and intermediate depths, while CQG-Single is stronger on Q2–Q3 through most of the curve; Q4 is nearly tied around $K = 90$–150. Beauty does not reproduce this rare-item shallow advantage; its paired seed-42 checkpoints favor CQG-Single under a user-level bootstrap at $K = 10, 50, 90$ (all $p < .001$), which does not capture training-seed uncertainty. For the seed-42 ML-1M user-level analysis, none of the aggregate paired differences at $K = 10, 50, 90$ is significant at the 0.05 level. Across the seven indexed training runs, however, CQG-Single exceeds TIGER in

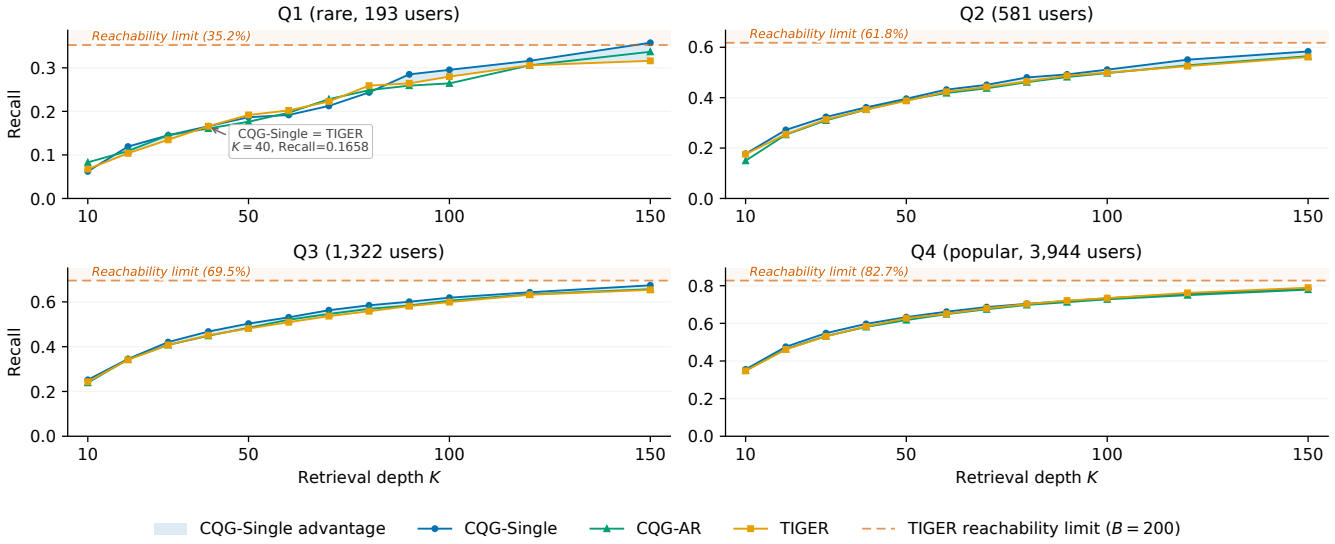

Figure 3: Measured per-quartile Recall@K curves on ML-1M, seed 42. CQG-Single and CQG-AR are shown separately; light-blue shading highlights intervals in which CQG-Single exceeds TIGER. Vermillion dashed lines show TIGER's user-weighted TargetSupport@200 within each quartile, which upper-bounds Recall@K for any ranking restricted to the same decoded beam; the light vermillion regions above them are unreachable for the evaluated model–beam pair. The Q1 annotation marks the exact measured CQG-Single–TIGER tie at $K = 40$ (Recall = 0.1658). Every curve point uses saved per-user output at $K \in \{10, 20, \ldots, 100, 120, 150\}$; no metric point is interpolated. All three methods return exactly 200 valid, distinct, history-filtered items per user.

Method	TargetInOutput @100	Q1 rare	Q2	Q3	Q4 popular	TargetItemSupport @200
TIGER	66.7%	25.8%	45.2%	54.0%	68.6%	78.1%
CQG-Single	67.3%	26.4%	46.9%	56.3%	70.0%	78.9%
CQG-AR	66.4%	24.4%	45.4%	54.8%	69.6%	78.8%

Table 2: ML-1M finite-output target support (seed 42), matched by effective returned-list size. TargetInOutput@100 is user weighted. Q1–Q4 use the same training-frequency item-to-quartile mapping as Figures 2 and 3, then compute item-balanced means over 162, 346, 584, and 781 distinct test-target items, respectively. TargetItemSupport@200 counts the fraction of all 1,873 distinct test-target items retrieved for at least one associated test instance. TIGER uses valid-prefix constrained decoding, exact code lookup, deterministic collision handling, and adaptive internal overgeneration; CQG uses dense Top-100 and Top-200 outputs. Every method returns exactly the stated number of valid, distinct, history-filtered items. These quantities are empirical finite-output support, not catalog-wide scoreability.

all seven, with a mean paired R@10 difference of 0.0107 (95% paired t-interval [0.0021, 0.0193]; exact sign-flip $p = .0156$; Appendix B). The first comparison quantifies within-checkpoint user-sampling uncertainty, whereas the second quantifies variation across indexed training runs. The corrected constrained Sports evaluations are still running and are marked pending in this review draft; results from the superseded decoding protocol are not mixed into the revised comparison. We therefore interpret crossover as an operating-point-dependent empirical pattern rather than a universal dominance claim.

MSE-trained candidate-generation and reranking diagnostics are reported separately in Appendix C.1; they are not results from the CE-trained CQG-Single or CQG-AR models used in the primary comparison.

Analysis

Objective visibility. Full-catalog CE gives continuous queries access to every item at each update, whereas SID decoding receives token-level supervision. To control this difference, we train CQG-AR with 256 sampled negatives. Sampled CE reduces R@10 from 0.2982 to 0.2895 while retaining performance close to the TIGER-style baseline. Thus training-time catalog visibility explains part, but not all, of the observed behavior; our architecture-level claim is limited to eliminating discrete code lookup and finite-beam exclusion. The sampled-negative control and broader frozen-item objective comparison are reported in Appendix F.1.

Scoreability versus empirical output coverage. Catalog-wide scoreability does not imply that ev-

Method	Config.	Params	ML-1M R@10	ML-1M NDCG@10	Beauty R@10	Sports R@10
SASRec	Full-rank	dataset-dependent	0.2797	0.1535	0.0558	0.0390
TIGER	Discrete AR	5.78M	0.2996 ± 0.0072	0.1739 ± 0.0046	0.0477 ± 0.0042	pending
CQG-Single	One-step	3.2M	0.3104 ± 0.0047	0.1822 ± 0.0044	0.0597 ± 0.0046	0.0363 ± 0.0015
CQG-AR	AR continuous ($L = 3$)	5.62M	0.2976 ± 0.0032	0.1735 ± 0.0016	0.0571 ± 0.0043	0.0355 ± 0.0004

Table 3: Cross-dataset primary comparison. SASRec entries are single-checkpoint full-rank baselines; TIGER, CQG-Single, and CQG-AR report mean ± sample SD. ML-1M uses seven training seeds (42–48), whereas Beauty and Sports use three (42–44); seed-level values and 95% t intervals are in Appendix B. SASRec parameter counts are dataset dependent because its item embedding table scales with catalog size. Each ML-1M TIGER checkpoint is reselected from the saved 10K–50K-step checkpoints using valid-prefix constrained validation; the corrected R@10 mean is unchanged from the earlier selection, while deeper returned-list metrics change. ML-1M and Beauty are full-user protocol matched. The Sports TIGER cell is marked pending until all three constrained runs finish; its CQG and SASRec cells are retained to make the scope of the pending update explicit. Amazon TIGER uses our hierarchical-k-means baseline.

ery item appears in a finite output set. With a matched returned-list size of 10 on ML-1M, TIGER, CQG-Single, and CQG-AR cover 70.1%, 69.7%, and 68.7% of the catalog, respectively. Thus, CQG-Single's higher mean R@10 is not explained by retrieving a larger union of distinct items at this cutoff. At Top-100, the two continuous methods cover 97.6% of the catalog, compared with 97.0% for TIGER. The separation is larger on Beauty at Top-10 (74.4%, 89.5%, and 87.9%), although all methods approach full union coverage by Top-100. These results reinforce the distinction among scoreability, per-user target support, and empirical catalog coverage. Corrected Sports TIGER coverage is pending in this review draft, so the previous cross-method Sports coverage interpretation is withheld.

Effect of Autoregressive Refinement. Under the controlled depth-ablation recipe with intermediate-loss weight 0.3, increasing CQG-AR depth from one to four steps does not monotonically improve shallow retrieval. R@10 varies only from 0.2909 to 0.2944, and the single-step boundary ($L = 1$) is competitive with the three-step control ($L = 3$). The final primary CQG-AR recipe instead uses intermediate-loss weight 0, so this experiment isolates depth only within the ablation recipe. Additional refinement is more visible at the deeper cutoff: R@100 increases from 0.6550 at $L = 1$ to 0.6697 at $L = 4$, although the trend is not monotonic at every intermediate depth. These results suggest that serial continuous refinement may adjust deeper candidate ordering without reliably improving shallow accuracy. We therefore use $L = 3$ primarily to match the three-level SID decoding structure, rather than because three continuous outputs are intrinsically necessary.

Returned-list-size sensitivity. For the seed-42 ML-1M TIGER checkpoint selected by constrained validation, increasing the effective returned-list size from 100 to 200 raises TargetInOutput from 66.7% to 76.3%. Across seven seeds, the corresponding means are 0.6667 ± 0.0018 and 0.7593 ± 0.0025. The evaluator increases the internal beam when deduplication and history filtering would otherwise leave too few items. Larger returned lists therefore relax the observed support limitation; our evidence supports a finite-compute trade-off, not an architecture-invariant catalog ceiling.

Discussion

Interpretation of the crossover. The observed crossover is a checkpoint-, popularity-, and operating-point-dependent empirical pattern rather than a universal ranking theorem. On ML-1M, TIGER is numerically stronger for rare targets at some shallow and intermediate cutoffs, whereas CQG-Single is generally stronger for more popular targets and at several deeper cutoffs; none of the aggregate paired ML-1M differences is significant at the 0.05 level. Beauty does not reproduce the rare-item shallow advantage, further limiting claims of universality. These results support a conditional tradeoff between concentrated finite-beam decoding and continuous retrieval, not universal dominance by either interface.

Practical implications. Continuous-query retrieval removes discrete code lookup and finite-beam exclusion, and can use exact scoring or ANN retrieval without changing the output dimensionality when encoded items are inserted. In the protocol-consistent comparison in Table 3, CQG-Single has the highest mean shallow accuracy, while CQG-AR shows that autoregressive modeling can be retained with continuous outputs. However, under the controlled depth-ablation recipe, additional continuous refinement does not monotonically improve shallow accuracy, and a single generated query remains competitive with three-step refinement. On Sports, full-rank SASRec remains strongest, showing that removing discrete output constraints does not by itself guarantee superior ranking accuracy. The resulting latency, memory, index-update cost, and cold-item accuracy have not been measured at production

Dataset	TIGER Cov.@10	TIGER Cov.@100	CQG-Single Cov.@10	CQG-Single Cov.@100	CQG-AR Cov.@10	CQG-AR Cov.@100
ML-1M	70.1%	97.0%	69.7%	97.6%	68.7%	97.6%
Beauty	74.4%	96.3%	89.5%	99.9%	87.9%	99.8%
Sports	pending	pending	63.1%	96.9%	71.8%	98.4%

Table 4: Returned-list-size-matched Catalog Coverage: empirical union-of-Top-K catalog coverage for seed 42. All datasets use the same effective returned-list size, full test-user sets, and history filtering across the three methods; denominators exclude padding and contain 3,416, 12,101, and 10,693 items for ML-1M, Beauty, and Sports. This output-union metric is distinct from per-user TargetInOutput and catalog-wide scoreability. Bold marks the highest empirical coverage at each cutoff within a dataset; it does not denote the most accurate recommender.

Depth	R@10	NDCG@10	R@100
$L = 1$	0.2924	0.1719	0.6550
$L = 2$	0.2909	0.1688	0.6525
$L = 3$	0.2914	0.1667	0.6619
$L = 4$	0.2944	0.1717	0.6697

Table 5: Single-seed CQG-AR decoding-depth ablation on ML-1M. All rows use seed 42, full-catalog CE, and intermediate-loss weight 0.3. Within this controlled ablation recipe, additional continuous refinements do not yield a monotonic R@10 or NDCG@10 improvement; the final primary recipe instead uses intermediate-loss weight 0.

scale.

Limitations

Our evidence is specific to the evaluated model–search–compute configurations. Wider beams substantially raise TIGER target support, and our hierarchical-k-means TIGER-style baseline is weaker than published SID systems. The closer T5/RQ-VAE Beauty reproduction also does not match the reference implementation's ranking accuracy, so its absolute coverage effect must not be generalized to a fully reproduced TIGER checkpoint. Matched-target analysis remains observational rather than a causal intervention on popularity, and the Beauty significance results use paired users from seed-42 checkpoints without capturing training-seed uncertainty. Full-catalog CE costs $O(N)$ per update and has only been evaluated on the reported catalog sizes; million-item training, ANN latency, memory, and dynamic-catalog accuracy remain open.

Conclusion

This paper studies finite-beam output support as a practical bottleneck in SID-based generative recommendation. In our evaluated models, target-conditioned support and empirical output coverage can be incomplete and popularity-skewed, and a checkpoint- and popularity-dependent crossover can emerge between SID decoding and vocabulary-free continuous retrieval. CQG-Single demonstrates a minimal one-step continuous query, while CQG-AR shows that an autoregressive decoder can replace discrete SID tokens with continuous residual outputs and retain competitive accuracy. Under the controlled depth-ablation recipe, a single generated query remains competitive with three-step refinement; this result is not asserted as a depth ablation of the final zero-intermediate-loss recipe. Together these results separate the value of autoregression from the constraints of a discrete output vocabulary and motivate protocol-matched evaluation of accuracy, target support, and returned-list-size-matched catalog coverage.

References

Covington, P.; Adams, J.; and Sargin, E. 2016. Deep Neural Networks for YouTube Recommendations. In Proceedings of the ACM Conference on Recommender Systems, 191–198.

Deng, J.; Wang, S.; Cai, K.; Ren, L.; Hu, Q.; Ding, W.; Luo, Q.; and Zhou, G. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. arXiv preprint arXiv:2502.18965.

Fu, J.; Ge, X.; Karatzoglou, A.; Arapakis, I.; Verberne, S.; Jose, J. M.; and Ren, Z. 2026. Differentiable Semantic ID for Generative Recommendation. In Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval.

Hidasi, B.; Karatzoglou, A.; Baltrunas, L.; and Tikk, D. 2016. Session-based Recommendations with Recurrent Neural Networks. In International Conference on Learning Representations.

Hou, Y.; Kim, H.; Ju, C. M.; Escoto, E.; Shah, N.; and McAuley, J. 2026. Expressiveness Limits of Autoregressive Semantic ID Generation in Generative Recommendation. arXiv preprint arXiv:2605.06331.

Huang, P.-S.; He, X.; Gao, J.; Deng, L.; Acero, A.; and Heck, L. 2013. Learning Deep Structured Semantic Models for Web Search using Clickthrough Data. In Proceedings of the ACM International Conference on Information and Knowledge Management, 2333–2338.

Kang, W.-C.; and McAuley, J. 2018. Self-Attentive Sequential Recommendation. In Proceedings of the IEEE International Conference on Data Mining, 197–206.

Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T. B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; and Amodei, D. 2020. Scaling Laws for Neural Language Models. arXiv preprint arXiv:2001.08361.

Lee, D.; Kim, C.; Kim, S.; Cho, M.; and Han, W.-S. 2022. Autoregressive Image Generation using Residual Quantization. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 11523–11532.

Li, J.; Zhang, W.; Wang, T.; Xiong, G.; Lu, A.; and Medioni, G. 2023. GPT4Rec: A Generative Framework for Personalized Recommendation and User Interests Interpretation. In SIGIR Workshop on Generative Information Retrieval.

Li, J.; Zhang, Y.; Bai, Y.; Zhu, S.; Xue, Z.; Zhao, X.; Wang, D.; Yang, F.; Rabinovich, A.; and He, X. 2026. UniGRec: Unified Generative Recommendation with Soft Identifiers for End-to-End Optimization. arXiv preprint arXiv:2601.17438.

Li, Z.; Sun, A.; and Li, C. 2023. DiffuRec: A Diffusion Model for Sequential Recommendation. In Proceedings of the ACM Web Conference.

Qu, H.; Lin, S.; Ding, Y.; Wang, Y.; and Fan, W. 2026. Diffusion Generative Recommendation with Continuous Tokens. In Proceedings of the ACM Web Conference.

Rajput, S.; Mehta, N.; Singh, A.; Keshavan, R. H.; Vu, T.; Heldt, L.; Hong, L.; Tay, Y.; Tran, V. Q.; Samost, J.; Kula, M.; Chi, E. H.; and Sathiamoorthy, M. 2023. Recommender Systems with Generative Retrieval. In Advances in Neural Information Processing Systems, volume 36, 10299–10315.

Sun, F.; Liu, J.; Wu, J.; Pei, C.; Lin, X.; Ou, W.; and Jiang, P. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In Proceedings of the ACM International Conference on Information and Knowledge Management, 1441–1450.

Tan, J.; Xu, S.; Hua, W.; Ge, Y.; Li, Z.; and Zhang, Y. 2024. IDGenRec: LLM-RecSys Alignment with Textual ID Learning. In Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval.

Tay, Y.; Tran, V. Q.; Dehghani, M.; Ni, J.; Bahri, D.; Mehta, H.; Qin, Z.; Hui, K.; Zhao, Z.; Gupta, J.; Schuster, T.; Cohen, W. W.; and Metzler, D. 2022. Transformer Memory as a Differentiable Search Index. arXiv preprint arXiv:2202.06991.

van den Oord, A.; Vinyals, O.; and Kavukcuoglu, K. 2017. Neural Discrete Representation Learning. arXiv preprint arXiv:1711.00937.

Wang, Y.; Hou, Y.; Wang, H.; Miao, Z.; Wu, S.; Sun, H.; Chen, Q.; Xia, Y.; Chi, C.; Zhao, G.; Liu, Z.; Xie, X.; Sun, H. A.; Deng, W.; Zhang, Q.; and Yang, M. 2022. A Neural Corpus Indexer for Document Retrieval. arXiv preprint arXiv:2206.02743.

Wang, Y.; Xun, J.; Hong, M.; Zhu, J.; Jin, T.; Lin, W.; Li, H.; Li, L.; Xia, Y.; Zhao, Z.; and Dong, Z. 2024. EAGER: Two-Stream Generative Recommender with Behavior-Semantic Collaboration. In Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 3245–3254.

Yang, L.; Paischer, F.; Hassani, K.; Li, J.; Shao, S.; Li, Z. G.; He, Y.; Feng, X.; Noorshams, N.; Park, S.; Long, B.; Nowak, R. D.; Gao, X.; and Eghbalzadeh, H. 2024. Unifying Generative and Dense Retrieval for Sequential Recommendation. arXiv preprint arXiv:2411.18814.

Yang, Z.; Wu, J.; Wang, Z.; Wang, X.; Yuan, Y.; and He, X. 2023. Generate What You Prefer: Reshaping Sequential Recommendation via Guided Diffusion. In Advances in Neural Information Processing Systems.

Zheng, B.; Hou, Y.; Lu, H.; Chen, Y.; Zhao, W. X.; Chen, M.; and Wen, J.-R. 2024. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. In Proceedings of the IEEE International Conference on Data Engineering, 1435–1448.

A Additional Cross-Dataset Diagnostics

We report completed, protocol-matched diagnostics on the two Amazon datasets used in the main paper. These results assess the robustness of our TIGER-style implementation; they are not presented as reproductions of the published TIGER architecture.

Dataset	Seed	R@10	N@10	R@30	R@90	TargetSupport@100
Beauty	42	0.0464	0.0262	0.0823	0.1392	14.55%
Beauty	43	0.0443	0.0261	0.0766	0.1299	13.71%
Beauty	44	0.0525	0.0295	0.0945	0.1568	16.40%
Sports	42	constrained test pending				
Sports	43	constrained test pending				
Sports	44	constrained test pending				

Table 6: TIGER-style SID diagnostics under constrained-validation checkpoint selection and valid-prefix decoding. Completed Beauty rows use the full 22,363 test users and exactly 100 valid, distinct, history-filtered items for TargetInOutput@100. Sports is explicitly marked pending in this review draft rather than mixing results from the superseded decoding protocol.

B Training-Seed Stability

Table 7 reports every completed ML-1M training seed under the final common protocol. TIGER uses valid-prefix constrained decoding, exact code lookup, and adaptive internal over-generation. Its mean TargetInOutput is 0.6667 ± 0.0018 at an effective returned-list size of 100 and 0.7593 ± 0.0025 at size 200. Across seven seeds, the 95% t intervals for R@10 are $[0.2930, 0.3063]$ for TIGER, $[0.3060, 0.3147]$ for CQG-Single, and $[0.2947, 0.3006]$ for CQG-AR. CQG-Single exceeds TIGER in all seven indexed runs. The mean paired seed-level R@10 difference is 0.0107 (95% paired t interval [0.0021, 0.0193]); an exact two-sided sign-flip permutation test over the seven paired differences gives $p = .0156$. These intervals quantify training-seed variation; they are distinct from the paired seed-42 user bootstrap in Appendix C.2.

All three methods use the same frozen SASRec item-embedding table, processed split, and evaluation history. For CQG-Single and CQG-AR, seeds 42–48 change downstream-model initialization, minibatch order, and stochastic optimization while leaving the item table fixed. For TIGER, the same indexed seed also changes hierarchical k-means initialization and SID-decoder training. Thus "paired seed" denotes matched replicate indices under a common item space and protocol; it does not imply that the three architectures consume identical random draws.

Seed	TIGER		CQG-Single		CQG-AR	
	R@10	N@10	R@10	N@10	R@10	N@10
42	0.2992	0.1756	0.3036	0.1736	0.2972	0.1720
43	0.3038	0.1751	0.3161	0.1874	0.3028	0.1750
44	0.2841	0.1644	0.3144	0.1831	0.2945	0.1711
45	0.3028	0.1768	0.3103	0.1830	0.3010	0.1755
46	0.3045	0.1781	0.3139	0.1849	0.2969	0.1739
47	0.2995	0.1728	0.3089	0.1832	0.2970	0.1734
48	0.3036	0.1747	0.3053	0.1804	0.2940	0.1738
Mean	0.2996	0.1739	0.3104	0.1822	0.2976	0.1735
Sample SD	0.0072	0.0046	0.0047	0.0044	0.0032	0.0016

Table 7: Seven-seed ML-1M results under valid-prefix constrained TIGER decoding. Each TIGER row uses the checkpoint reselected by constrained validation and evaluates all 6,040 test users with the same history filtering. N@10 denotes NDCG@10.

Preliminary Sports multimodal diagnostic. We additionally ran a single-seed diagnostic in which CQG-Single, CQG-AR, and the TIGER-style decoder share item embeddings formed from collaborative features and ImageNet-pretrained ResNet-18 image features. Images were available for only 5,268 of 10,693 non-padding catalog items; items without an image use the collaborative component with a zero image feature. Because image availability is incomplete and no multi-seed estimate is available, this experiment is preliminary and is not used to support the main claims.

Method	R@10	NDCG@10	R@90	TargetInOutput@100
CQG-Single (multimodal, seed 42)	0.0326	0.0174	0.1154	–
CQG-AR (multimodal, seed 42)	0.0296	0.0161	0.1087	11.62%
TIGER-style SID (multimodal, seed 42)	0.0254	0.0137	0.0972	10.06%

Table 8: Preliminary single-seed Sports multimodal diagnostic. CQG-Single, CQG-AR, and the TIGER-style SID model use the same fused item space; the result is not directly comparable to a full-coverage multimodal benchmark because only 49.3% of catalog items have images.

C Experimental Protocol and Reproducibility

C.1 MSE-Trained Retrieve-and-Rerank Diagnostics

We evaluate a matched 100-candidate retrieve-and-rerank pipeline using MSE-trained MSE-trained continuous-query checkpoints. Across ML-1M, Beauty, and Sports, CQG-Single followed by SASRec reranking reaches 97.7–102% of full-rank SASRec Recall@10; the completed CQG-AR controls on Beauty and Sports are also reported in Table 9. Table 10 further compares the learned retriever with random, last-item kNN, and mean-of-five kNN baselines under the same SASRec reranker. These candidate-generation diagnostics are separate from the CE-trained standalone results in Table 3.

Method	ML-1M	Beauty	Sports
Recall@10			
SASRec	0.2797	0.0558	0.0390
CQG-Single MSE standalone	0.2425	0.0524	0.0307
CQG-Single MSE + reranker	0.2801	0.0569	0.0381
CQG-AR MSE standalone (seed 42)	0.2414	0.0466	0.0294
CQG-AR + reranker (seed 42)	0.2791	0.0486	0.0378
TIGER candidates + SASRec	0.2864	–	–
NDCG@10			
SASRec	0.1535	0.0296	0.0197
CQG-Single MSE standalone	0.1424	0.0266	0.0150
CQG-Single MSE + reranker	0.1554	0.0308	0.0194
CQG-AR MSE standalone (seed 42)	0.1366	0.0276	0.0160
CQG-AR + reranker (seed 42)	0.1543	0.0265	0.0194
TIGER candidates + SASRec	0.1571	–	–

Table 9: Matched 100-candidate retrieve-and-rerank diagnostic. CQG-Single rows use the MSE-trained retriever; CQG-AR uses the final available seed-42 candidate outputs for each dataset. The TIGER reranking control is available on ML-1M only. These rows are separate from the standalone primary comparison.

Retriever	ML-1M R@10	Sports R@10
Random	0.0252	0.0037
Last-item kNN	0.2349	0.0311
Mean-of-5 kNN	0.2545	0.0310
CQG-Single MSE + SASRec	0.2801	0.0381
CQG-AR + SASRec	0.2791	0.0378
SASRec full-rank	0.2797	0.0390

Table 10: Retrieval-quality hierarchy. All methods except SASRec full-rank use SASRec reranking over top-100 candidates.

C.2 Paired Crossover Values

		R@10		R@50		R@90	
Quartile	Users	CQG-Single	TIGER	CQG-Single	TIGER	CQG-Single	TIGER
Q1 rare	193	0.0622	0.0674	0.1865	0.1917	0.2850	0.2642
Q2	581	0.1773	0.1756	0.3959	0.3873	0.4923	0.4871
Q3	1,322	0.2519	0.2436	0.5030	0.4811	0.6006	0.5809
Q4 popular	3,944	0.3555	0.3474	0.6331	0.6273	0.7186	0.7208
Aggregate	6,040	0.3063	0.2992	0.5675	0.5583	0.6571	0.6531

Table 11: Paired seed-42 ML-1M diagnostic using the same 6,040 users and exactly 200 valid, distinct, history-filtered items from constrained TIGER decoding and CQG-Single. Aggregate CQG-minus-TIGER Recall differences are 0.0071 at $K = 10$ (95% CI $[-0.0025, 0.0169]$, $p = .149$), 0.0093 at $K = 50$ (CI $[-0.0003, 0.0189]$, $p = .060$), and 0.0040 at $K = 90$ (CI $[-0.0048, 0.0127]$, $p = .397$), from 10,000 paired user-bootstrap resamples. None is significant at the 0.05 level.

C.3 Computing Infrastructure

Experiments ran in a Linux container with one NVIDIA A800 80GB PCIe GPU (81,920 MiB device memory), 14 allocated CPU cores, and approximately 120GB of host-visible memory. The software environment uses Python 3.12.3, PyTorch 2.8.0+cu128, CUDA 12.8 as bundled with PyTorch, cuDNN 9.10.2, Transformers 5.13.1, NumPy 2.3.2, scikit-learn 1.9.0, and FAISS-CPU 1.14.3. Training uses bfloat16 automatic mixed precision where indicated; evaluation metrics are accumulated outside autocast.

C.4 Dataset Splits

All datasets are converted into chronological user sequences. For each user, we use the last interaction for testing, the penultimate interaction for validation, and all earlier interactions for training. Users with insufficient history after filtering are removed. Evaluation is full-catalog unless otherwise stated: the target item is ranked against all candidate items rather than against sampled negatives. The reported top-K metrics use $K \in \{5, 10, 20, 30, 50, 70, 90\}$ when available.

C.5 Evaluation Metrics

For a user u with held-out target item $i_u^\star$, Recall@K is one if $i_u^\star$ appears in the top-K list and zero otherwise. NDCG@K discounts a hit by the target rank. We distinguish two output-based support metrics. The SID target-support rate is

$$\text{TargetSupport@B} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbf{1}\{i_u^\star \in \text{SIDOutput}_B(u)\},$$

where $\text{SIDOutput}_B(u)$ is the final ranked list of at most B valid, distinct, history-filtered items produced by constrained SID decoding. When filtering would leave fewer than B items, the internal beam is expanded until the list is filled or the valid search space is exhausted. Completed sequences require an exact catalog-code match; no numeric nearest-code mapping is used. This is a per-user target-conditioned quantity and is closely related to Recall@B; it must not be interpreted as catalog coverage. For a common finite-output comparison across methods, we define

$$\text{TargetInOutput@K}(M) = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbf{1}\{i_u^\star \in \text{TopK}_M(u)\}.$$

This common finite-output measure is Recall@K under our single-target leave-one-out protocol. Let $\mathcal{I}_{\text{test}} = \{i_u^\star : u \in \mathcal{U}\}$ be the set of distinct held-out target items. We additionally define

$$\text{TargetItemSupport@K}(M) = \frac{|\{i \in \mathcal{I}_{\text{test}} : \exists u,\ i_u^\star = i \ \wedge \ i \in \text{TopK}_M(u)\}|}{|\mathcal{I}_{\text{test}}|}.$$

Unlike user-weighted Recall, this measures how many distinct target-item types are successfully retrieved at least once. For any method M, empirical union coverage at a matched returned-list size K is

$$\text{CatalogCoverage@K}(M) = \frac{\left|\bigcup_{u \in \mathcal{U}} \text{TopK}_M(u)\right|}{|\mathcal{I}|}.$$

We report this same output-set quantity for TIGER, CQG-Single, and CQG-AR. Catalog-wide scoreability of a dense interface is a separate property and does not imply 100% CatalogCoverage@K.

C.6 Matched-Target Resampling Diagnostic

For the matched-target diagnostic, we first group distinct test-target items by training-frequency quartile. Let $\mathcal{Q}_q^{\text{test}}$ be the test targets in quartile q and let $m = \min_q |\mathcal{Q}_q^{\text{test}}|$. For each trial, we uniformly sample $S_q \subseteq \mathcal{Q}_q^{\text{test}}$ with $|S_q| = m$ and compute:

$$\text{MatchedTargetSupport}_q = \frac{1}{m} \sum_{i \in S_q} \mathbf{1}\{i \text{ is supported for its associated test instance}\}.$$

We report the mean and standard deviation over 200 trials. This equalizes the number of distinct sampled targets, but it is not a substitute for the matched CatalogCoverage@K measurement above.

Our primary diagnostics directly report item-balanced support in Table 2, matched union coverage in Table 4, and explicit training-seed variation in Table 7. These quantities separately address target imbalance, output diversity, and training-run uncertainty; within-seed item resampling is not used as a proxy for training-seed variation.

D Model and Training Details

D.1 Frozen Collaborative Item Space

SASRec backbone and examples. The item space is learned by a SASRec encoder trained from scratch on the training portion of each dataset; no text, image, T5, or language-model features are used in the main item encoder. The model contains a trainable item lookup table $V \in \mathbb{R}^{N \times 64}$, learned positional embeddings, and two causal self-attention blocks with two heads, hidden dimension 64, and dropout 0.2. For a chronological training sequence $(i_1, \ldots, i_T)$, every position $t = 2, \ldots, T$ yields

$$([i_{\max(1,t-L)}, \ldots, i_{t-1}],\, i_t),$$

where $L = 50$ on ML-1M and $L = 20$ on Beauty and Sports. Histories are left-padded with item ID 0; the padding embedding is fixed at zero and is excluded from evaluation.

BPR pretraining and checkpoint selection. Let h_t be the final SASRec hidden state for a history, i_t its positive next item, and i^- one item drawn uniformly from IDs $\{1, \ldots, N-1\}$. SASRec is optimized with

$$\mathcal{L}_{\text{BPR}} = -\log \sigma(h_t^\top V_{i_t} - h_t^\top V_{i^-}).$$

We use Adam with learning rate 10^{-3} and batch size 256. Every five epochs, validation ranks the held-out validation item against the full catalog (masking padding). We retain the checkpoint with the highest validation NDCG@10. The maximum is 200 epochs for ML-1M and Beauty and 400 for Sports; checkpoint selection, rather than the last epoch, determines the exported representation.

Export, normalization, and sharing. After loading the selected checkpoint, we export the lookup parameters directly,

$$E = \texttt{SASRec.item\_emb.weight} \in \mathbb{R}^{N \times 64}.$$

Thus e_i is a global collaborative item parameter, not a contextual SASRec hidden state. We freeze E for every downstream run. Its non-padding rows are L2-normalized for cosine retrieval, CE, BPR, and InfoNCE; the MSE-trained CQG-Single objective regresses to the raw exported vector. The same frozen table is used (i) to embed history IDs, (ii) as the MSE/InfoNCE target table or the full-catalog CE classifier, (iii) as the exact or ANN retrieval index, and (iv) as the input to the hierarchical clustering used by our TIGER-style baseline. This sharing prevents differences in the item encoder from explaining differences among the output interfaces. Dataset-specific seeds use their corresponding SASRec export where stated; the canonical ML-1M seed-42 table has shape 3417×64.

D.2 CQG-Single Architecture and Optimization

CQG-Single has no pretrained backbone and does not update E. Frozen 64-dimensional history vectors are projected to width 256 and combined with learned positional embeddings. Four pre-norm causal Transformer blocks, each with four attention heads, feed-forward dimension 1024, and dropout 0.1, produce the final valid history state. A linear head maps this state back to 64 dimensions and the query is L2-normalized before scoring the normalized catalog. The CE configuration uses Adam, learning rate 10^{-3}, batch size 256, at most 50 epochs, validation every five epochs, and early stopping with patience 15 based on validation NDCG@10. MSE and InfoNCE change only the objective and their stated learning-rate/epoch settings in Table 12; the history protocol and frozen target table remain the same.

D.3 CQG-AR Architecture and Optimization

CQG-AR is also trained from scratch with frozen E. It projects history vectors from 64 dimensions to width 384 and uses four pre-norm causal Transformer blocks with six heads, feed-forward dimension 1024, and dropout 0.1. A learned BOS vector starts continuous decoding. At step ℓ, a linear head followed by tanh produces $\Delta_\ell \in \mathbb{R}^{64}$; a learned scalar controls its magnitude, the cumulative query is initialized as $q^{(0)} = \mathbf{0}$, and

$$q^{(\ell)} = \mathrm{norm}(q^{(\ell-1)} + \Delta_\ell).$$

The residual is projected back to width 384, tagged with a generated-token type embedding and a position embedding, and appended as the next causal decoder input. The main configuration uses three steps, so the serial outputs are $(\Delta_1, \Delta_2, \Delta_3)$, but all three refer to the same next-item target i_t; they are not labels for three SID levels.

For the final CE recipe, only $q^{(3)}$ receives direct catalog CE supervision (`intermediate_weight=0`); earlier steps receive gradient through their effect on later decoding. The depth ablation instead uses intermediate weight 0.3 for the non-final queries, always with the same next-item label. CQG-AR uses AdamW with learning rate 3×10^{-4}, weight decay 0.01, batch size 1024, at most 30 epochs, bfloat16 automatic mixed precision, gradient-norm clipping at 1.0, and early stopping with patience 5 on validation NDCG@10. The selected checkpoint is evaluated against the full catalog after appending the validation item to the test history and masking all previously observed items.

D.4 Hyperparameters

Method	Item d	Hidden	Layers	Heads	LR	Batch	Epochs/Steps	Objective
SASRec item encoder	64	64	2	2	1e-3	256	200/400 max.	BPR, one uniform negative
CQG-Single MSE	64	256	4	4	1e-3	256	50 max.	MSE to target embedding
CQG-Single InfoNCE	64	256	4	4	1e-4	256	100 max.	In-batch InfoNCE, $\tau = 0.07$
CQG-Single CE	64	256	4	4	1e-3	256	50 max.	Full-catalog softmax CE
CQG-AR	64	384	4	6	3e-4	1024	30 max.	AR residuals + full-catalog CE
CQG-AR sampled CE	64	384	4	6	3e-4	1024	30 max.	AR residuals + 256 negatives
Qwen hybrid probe	64	–	–	–	5e-5	32×8	10	InfoNCE with projection head
TIGER-style SID decoder	64	384	4	6	1e-3	256	50K steps	Autoregressive CE over SID tokens
TIGER T5/RQ-VAE reproduction	–	–	–	–	1e-4	256	200 max.	Autoregressive CE over SID tokens

Table 12: Training hyperparameters for the three datasets. "Hidden" is the Transformer width; all continuous queries remain 64-dimensional. SASRec uses at most 200 epochs on ML-1M/Beauty and 400 on Sports. Sequence length is 50 for ML-1M and 20 for Beauty/Sports; "max." denotes early-stopped training.

D.5 Continuous-Query Dataloader

CQG-Single and CQG-AR use the same next-item examples derived from user histories. Each training example contains a padded history sequence and the next-item target. The history item IDs are mapped to frozen item embeddings before entering the query generator; the target ID is used either to index the target embedding for MSE/InfoNCE or to index the full-catalog label for CE. A batch therefore has the form

$$(\mathbf{H}, \mathbf{y}) = \{([i_{t-L}, \ldots, i_{t-1}], i_t)\}_{b=1}^{B},$$

where L is the maximum sequence length and B is the batch size. Padding positions are masked and never treated as candidate items. This dataloader is intentionally the same sequential next-item protocol used by SASRec, so the comparison isolates the output interface rather than changing the recommendation task.

For example, one actual ML-1M training pair is

$$([2758, 1069, 1445, 862, 1979],\ 1520).$$

The model input is the 5×64 matrix obtained by looking up the five history IDs in frozen E. Under CE, the label is the integer 1520 among 3,416 non-padding catalog items; the implementation stores these items in a 3,417-row table whose row 0 is masked padding. Under MSE or InfoNCE, the corresponding target is row E_{1520}. At test time the validation item is appended to the training history, the final item is the held-out target, and all items already present in the resulting history are masked from ranking.

D.6 Continuous Query Training Objectives

The MSE variant regresses the generated query q_u to the frozen target embedding $e_{i^\star}$ (the MSE-trained CQG-Single run uses the raw SASRec export):

$$\mathcal{L}_{\mathrm{MSE}} = \|q_u - e_{i^\star}\|_2^2.$$

The InfoNCE variant treats other targets in the batch as negatives:

$$\mathcal{L}_{\mathrm{NCE}} = -\log \frac{\exp(q_u^\top e_{i^\star}/\tau)}{\sum_{j\in\mathcal{B}} \exp(q_u^\top e_j/\tau)}.$$

The CE variant scores the full frozen catalog:

$$\mathcal{L}_{\mathrm{CE}} = -\log \frac{\exp(q_u^\top e_{i^\star}/\tau)}{\sum_{j=1}^{N} \exp(q_u^\top e_j/\tau)}.$$

For CQG-AR, $q_u = q^{(3)}$ in the final recipe. All autoregressive steps share the same next-item target; there is no separate label per residual. All variants can score every encoded item by dense similarity at inference time. Their empirical CatalogCoverage@K is nevertheless measured from the union of finite top-K outputs.

E SID Construction and Decoder Training

E.1 Semantic-ID Construction

The SID baseline follows the TIGER-style pipeline. First, a SASRec encoder is trained on the same chronological sequences used by both continuous methods. Its learned item embedding matrix is frozen and clustered by hierarchical k-means. Each item receives a depth-3 code path. The resulting tuple of cluster indices is treated as the target token sequence for the autoregressive decoder. Code collisions are resolved by assigning residual tie-break identifiers when needed, but in the reported ML-1M run collisions are negligible: 99.4% of items receive unique codes.

E.2 TIGER T5/RQ-VAE Reproduction Recipe

Our closer Beauty reproduction loads the available precomputed SID array `Beauty_t5_rqvae.npy`; the three decoder seeds do not retrain the RQ-VAE tokenizer. The file assigns a distinct complete code to each of 12,101 items in the corresponding Beauty data directory. We train the provided T5-style encoder–decoder separately with seeds 42, 43, and 44. Each run uses 200 maximum epochs, batch size 256, learning rate 10^{-4}, inference batch size 96, beam width 30, and early stopping after ten validation checks without improvement in NDCG@20. The model has approximately 4.59M trainable parameters. Its input is the serialized user interaction history and its target is the next item's multi-token SID; training minimizes teacher-forced autoregressive token cross-entropy. At inference, beam search generates SID sequences that are mapped back to catalog items.

The exact command template is:

```
python main.py -dataset_path ../data/Beauty
-code_path ../data/Beauty/Beauty_t5_rqvae.npy
-num_epochs 200 -batch_size 256 -infer_size 96
-lr 1e-4 -beam_size 30 -seed {42,43,44}.
```

In addition to our hierarchical-k-means diagnostic baseline, this closer Beauty SID recipe uses the available precomputed T5/RQ-VAE codes and a T5 encoder–decoder. It does not match the reference implementation's reported ranking accuracy, so we do not use published SID numbers as main-text benchmarks.

Output-set coverage. We also decode the full 22,363-user Beauty test set from each T5/RQ-VAE checkpoint with beam width 100 and map valid generated codes back to catalog items. Across seeds, 99.48% of generated sequences are valid SIDs, but the union covers only 44.0% ± 13.7% of the 12,101-item catalog. The popularity gradient is consistent despite substantial seed variance: mean coverage rises from 14.0% ± 7.6% in Q1 to 88.7% ± 13.0% in Q4 (Table 13). Because this reproduction has an unresolved ranking-accuracy gap, it shows only that the coverage pattern is not unique to our hierarchical-k-means code assignment; it is not a substitute for measuring the reference implementation's checkpoint.

Catalog group	Coverage@100	Std.
Q1 rare	14.0%	7.6%
Q2	26.1%	13.8%
Q3	47.4%	20.3%
Q4 popular	88.7%	13.0%
Overall	44.0%	13.7%

Table 13: Empirical union-of-top-100 catalog coverage for the closer T5/RQ-VAE Beauty reproduction, averaged over decoder seeds 42–44. Each quartile contains approximately one quarter of the catalog.

This recipe yields Beauty R@10 0.0408 ± 0.0016, whereas the reference implementation reports a higher value (Table 14). We do not treat that published value as a benchmark in the main experiments because the comparison is not protocol matched. Plausible, non-exclusive sources of the remaining gap are:

- Code and data snapshot. We use an available precomputed SID array and its paired Beauty directory; these may not be byte-identical to the tokenizer checkpoint and processed split used for the reported result.
- Item features and tokenizer training. The precomputed codes prevent us from verifying or retraining the exact content-feature extraction, RQ-VAE initialization, codebook optimization, and collision handling pipeline.
- Model and optimization recipe. Decoder scale, initialization, batch construction, learning-rate schedule, regularization, stopping criterion, and checkpoint selection can all affect ranking accuracy.
- Constrained decoding. Trie construction, invalid-code filtering, beam width, length normalization, duplicate removal, and the mapping from completed codes to items need not match the reference implementation exactly.
- Evaluator protocol. We use our leave-one-out, historical-item filtering, and full-catalog evaluator. Differences in history construction, candidate filtering, item indexing, and metric implementation can change the reported values.

Because these factors change together, the present experiment cannot identify a single causal source of the gap. We therefore report the discrepancy for transparency, avoid tuning against the published test number, and label all measurements from this reproduction separately.

Method	Beauty R@10	Beauty NDCG@10
Ours: TIGER-style	0.0477 ± 0.0042	0.0273 ± 0.0019
TIGER (Rajput et al. 2023)	0.0648	0.0384
LETTER-TIGER (Zheng et al. 2024)	0.0672	0.0364
EAGER (Wang et al. 2024)	0.0836	0.0525

Table 14: Published and reproduced SID results on Amazon Beauty, provided only to document the implementation gap. Our row is a three-seed mean and sample standard deviation under our evaluator. Published rows are transcribed from their respective papers and are not controlled benchmarks: tokenizers, item features, model architectures, data processing, decoding, and evaluation protocols differ.

E.3 TIGER-Style Dataloader

The SID decoder dataloader uses the same history-target pairs as CQG-Single and CQG-AR, but replaces the target item ID with its code tuple. Each batch contains a padded item-history sequence and a target SID sequence. Training minimizes token-level cross-entropy over the code positions.

At inference, we construct a trie from the catalog SID tuples. The first decoding step permits only catalog-valid first-level codes; each later step permits only tokens that extend the current prefix to at least one catalog code. Completed tuples are resolved by exact lookup only; no numeric nearest-code mapping is used. Items sharing a tuple receive a deterministic item-ID tie break. Previously observed items are removed, and the internal beam is doubled when necessary until the requested number of valid, distinct, history-filtered items is available (up to the documented safety cap). For ML-1M Top-200 evaluation, all 6,040 users receive exactly 200 items; the mean internal beam is approximately 405 and the maximum used is 800.

Checkpoint selection follows the same decoding protocol. For every ML-1M seed, we evaluate the saved 10K, 20K, 30K, 40K, and 50K-step checkpoints on the fixed 500-user validation subset, select by constrained-validation R@10, and then evaluate the selected checkpoint on the complete test population.

F Additional Ablations

F.1 Frozen-Item Objective Diagnostics

Objective	R@10	NDCG@10	R@100
Full-catalog CE	0.2982 ± 0.0042	0.1727 ± 0.0020	0.6631 ± 0.0048
Sampled CE (256 negatives)	0.2895 ± 0.0077	0.1663 ± 0.0034	0.6592 ± 0.0104

Table 15: ML-1M objective-visibility control (mean $\pm$ sample SD; seeds 42–44). Reducing catalog visibility to 256 sampled negatives lowers R@10 by 0.0087 but retains competitive accuracy, so objective visibility explains part, not all, of the continuous model's behavior.

Under the corrected leave-one-out evaluator, full-catalog CE yields higher Recall@10 than MSE and in-batch InfoNCE in the frozen item space (Table 16). Figure 4 provides a broader diagnostic across the evaluated objectives, including CE-joint for completeness. CE-joint is not a controlled loss-only comparison because it changes both the objective and the item representations.

Objective	Test R@10	vs. SASRec
SASRec BPR	0.2797	100.0%
CQG-Single full-catalog CE	0.3114	111.3%
CQG-Single MSE	0.2425	86.7%
CQG-Single InfoNCE	0.2386	85.3%
CQG-AR full-catalog CE	0.2982	106.6%
CQG-AR MSE	0.2424	86.7%
CQG-AR InfoNCE	0.2324	83.1%

Table 16: Frozen-item objective landscape on ML-1M. CQG-Single CE/MSE and all CQG-AR rows are three-seed means under the corrected test-history protocol; CQG-Single InfoNCE is an auxiliary diagnostic.

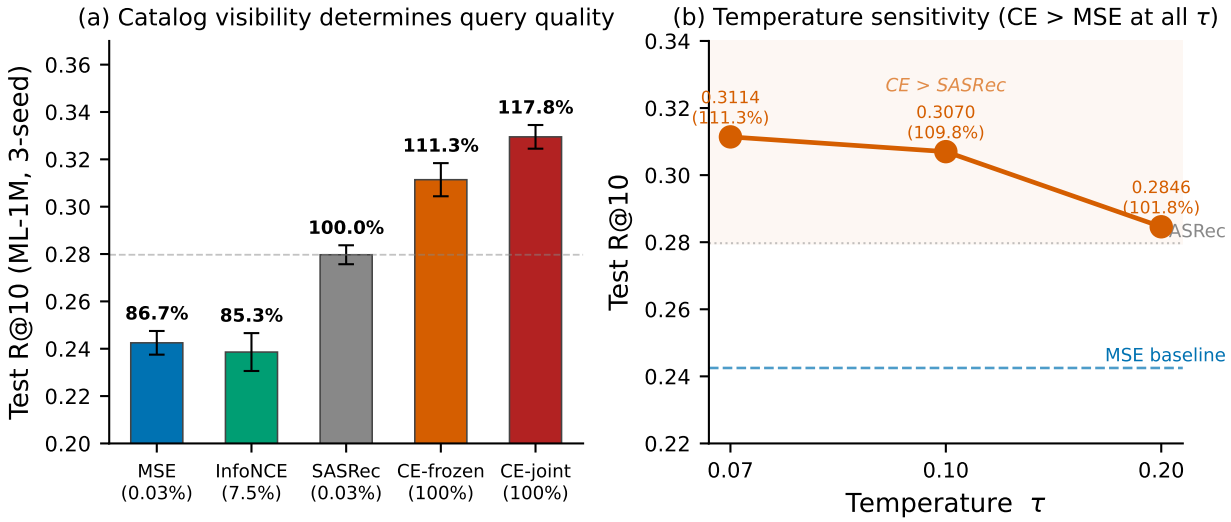

Figure 4: Auxiliary objective and item-space diagnostic on ML-1M. MSE, InfoNCE, and full-catalog CE in Table 16 keep the SASRec item embedding space frozen. CE-joint updates the item representations and is shown only as a separate diagnostic, not as a controlled loss-only comparison.

F.2 Temperature Ablation

Objective	Test R@10
CQG-Single CE, $\tau = 0.07$	0.3114
CQG-Single CE, $\tau = 0.10$	0.3070
CQG-Single CE, $\tau = 0.20$	0.2846
CQG-Single MSE	0.2425
CQG-AR CE, $\tau = 0.07$	0.2972
CQG-AR CE, $\tau = 0.10$	0.2886
CQG-AR CE, $\tau = 0.20$	0.2725

Table 17: Temperature ablation on ML-1M under the corrected test-history protocol. CQG-Single values are three-seed means; CQG-AR values are aligned seed-42 controls. Lower temperatures improve shallow retrieval in the tested range.

G LLM Backbone Diagnostic

We tested whether a pretrained LLM backbone improves continuous query generation. Multiple Qwen configurations across three paradigms produced a negative but informative result (Figure 5). Shared dual-tower training collapsed item representations, with item–item cosine similarity increasing toward 0.9936. A frozen-item hybrid avoided item collapse but produced loss–metric decoupling: InfoNCE loss improved while Recall@10 remained far below the SASRec gate. The best text-aware hybrid reached R@10 = 0.2118 on ML-1M but still trailed the single-run, MSE-frozen scratch Transformer used in this diagnostic (R@10 = 0.248). This checkpoint is distinct from the MSE-trained CQG-Single three-seed result of 0.2425 in Table 16. This result does not imply that LLMs cannot support recommendation; it shows that pretrained language representations are not automatically aligned with the frozen collaborative item space.

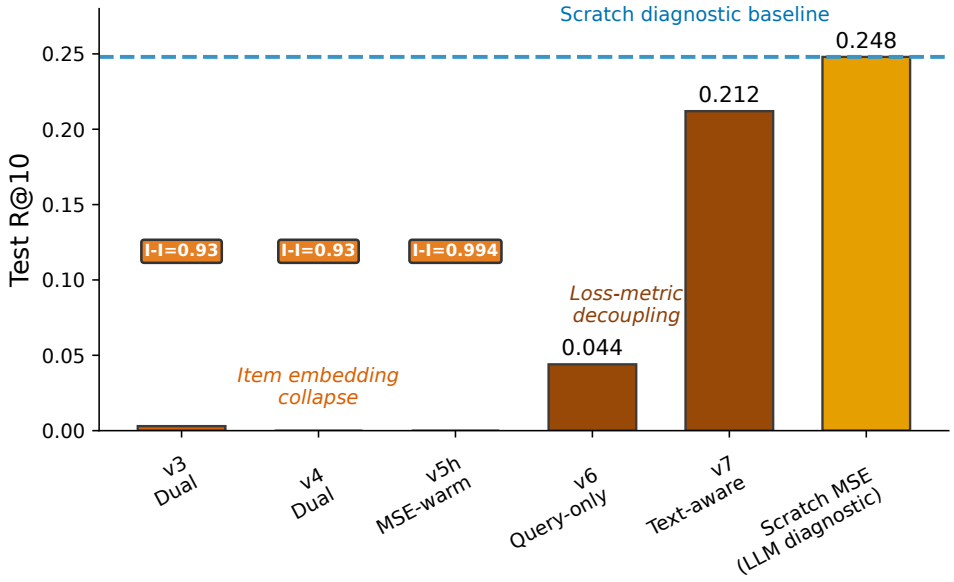

Figure 5: LLM backbone diagnostics. Pretrained language backbones do not automatically align with the frozen collaborative item space; the best text-aware Qwen hybrid still trails the single-run, MSE-frozen scratch query Transformer used in this diagnostic (R@10 = 0.248). This checkpoint is separate from the three-seed CQG-Single MSE result in Table 16.

Model	R@5	R@10	R@20	R@50	R@70	R@90
Scratch Transformer (LLM diagnostic)	0.172	0.248	0.345	0.498	0.555	0.595
Qwen text-aware hybrid	0.134	0.212	0.308	0.460	0.517	0.564

Table 18: Single-run recall curve comparison between the MSE-frozen scratch Transformer used in the LLM diagnostic and the best tested LLM hybrid configuration. The scratch row is not the three-seed CQG-Single MSE aggregate reported in Table 16.

The full recall curve confirms the diagnostic: the best tested LLM hybrid underperforms the scratch Transformer at every reported depth.

H Tokenization Diagnostics

Several neural RQ-VAE configurations collapsed to degenerate codebooks with less than 10% utilization. A k-means fallback achieved high code utilization but showed large downstream variance across decoder configurations. We do not use these failures as evidence that SID methods are generally weak; instead, they motivate careful baseline validation against published TIGER, LETTER, and EAGER results.

Configuration	Utilization	Outcome
Neural RQ-VAE variants	<10%	Codebook collapse
k-means fallback A	99.8%	R@10 = 0.1230, 15% reachability
k-means fallback B	99.8%	R@10 = 0.1932, 52% reachability

Table 19: Tokenization diagnostics. Neural RQ-VAE collapsed in the tested configurations; k-means improved utilization but remained sensitive to decoder training and beam-search support.

I Dynamic Catalog Discussion

CQG-Single and CQG-AR share the same index-level update interface. A new encoded item can be inserted into the ANN index without adding an output token, rebuilding a code trie, or changing either model's output dimensionality. This interface property does not guarantee cold-item ranking quality. In contrast, a SID system must assign the new item to a code path; if the assignment changes the code distribution or causes collisions, the codebook, trie, and decoder may need to be refreshed. A direct dynamic-catalog evaluation would hold out a fraction of items during training, insert them into the retrieval index only at test time, and measure new-item Recall@K, warm-item Recall@K, index update cost, and whether model retraining is required.